Gated replans in a study planner with BKT tracing

Rohit Saji[1*] and Ramesh Prakash Guledgudd[1]

[1*]Department of Data Science and Artificial Intelligence, PES University, 100 Feet Ring Road, Banashankari 3rd Stage, Bengaluru, 560085, Karnataka, India.

*Corresponding author(s). E-mail(s): iam.rohitsaji@gmail.com;
Contributing authors: rameshpg1988@gmail.com;

Abstract

Fixed study plans often fail within days. Time logs record hours without measuring understanding. This paper closes the loop with assessment-based proof of learning. The system generates grounded practice from learner materials in one family per call. A server grades each family with its own arm. Bayesian Knowledge Tracing tracks mastery.

Reranked retrieval grounds each question in cited chunks. A verdict gate replans only when proof holds. The gate uses Jev with approve at 0.7. Retrieval on the frozen 754-chunk corpus increases from 0.60 to 0.67 at r@1 and from 0.80 to 0.90 at r@3 with rerank. Groundedness on the 30-item fresh judge is 26 of 30 (0.867) with near-duplicate rate 0.004. Live BKT reaches AUC 0.74 and ties the running-proportion baseline while it halves calibration error.

The gated loop converts 4 of 4 synthetic longitudinal cases to replans. The stack runs Qwen3 embeddings plus rerank on demand with Piston grading. Fixed budgets appear in Methods. Live runs show zero console errors across objectives, rubrics, coding, guides, practice, feedback, mastery, and ingestion. The scope stays single-learner with synthetic validation. Phase I measured change detection as a robust null; Phase II ships no detector, and split-conformal intervals stay unshipped.

Keywords: Closed-loop tutoring, Bayesian Knowledge Tracing (BKT), reranked retrieval, grounded generation, verdict-gated replanning

1 Introduction

Self-directed learners commit to a fixed plan and often leave the plan within days. The plan does not adapt, and logged hours do not show whether study produced understanding. Phase I of this work built an open-loop planner that estimates true pace from session history. It regenerates the plan when the projection diverges from the deadline. The open loop adapts to pace but never checks learning.

Phase II closes the loop with assessment-based proof. The system generates grounded practice from learner materials. An authoritative arm grades each answer, Bayesian Knowledge Tracing tracks mastery, and a Socratic ladder guides the learner before the loop verdict replans. This paper reports the deployable loop and its measured proof.

Methods states each fixed budget with its frozen evidence, and Results reports coherent tables from one corpus. What is still missing is a loop that turns a graded attempt into a replan decision without overclaiming. The research question is whether a gated, evidence-backed loop can replan from grades while refusing what it cannot ground. The contribution is the loop itself: one-family grounded generation, per-family server grading, BKT tracing, reranked retrieval, and a verdict gate, all evaluated on frozen evidence. One-to-one tutoring outperforms group instruction [1]; this paper tests the single-learner end of that claim with synthetic cases, not a classroom.

Three rules set the scope. Every number traces to frozen evidence or live code. Synthetics replace a classroom study, and the text says so wherever it matters. Rejected arms and nulls appear beside positive results.

2 Literature Review

Each entry names the source, its mechanism, its metric, and the gap it leaves. Table 1 shows the selected and rejected options per stream.

2.1 Tracing and mastery

BKT tracks a per-skill mastery chance through four fitted terms: prior $P(L_0)$, learn rate T, guess G, and slip S [2]. After each answer the posterior updates from the response and then moves toward mastery by the fitted rate:

$$P_{\text{correct}}(L_n) = \frac{P(L_{n-1})(1-S)}{P(L_{n-1})(1-S) + (1-P(L_{n-1}))G},$$
$$P(L_n) = P_{\text{obs}}(L_n) + (1 - P_{\text{obs}}(L_n))T. \tag{1}$$

The first term changes the estimate from the observed response. The second term moves the learner toward mastery by the fitted rate.

Chen et al. test a hierarchical Bayesian teaching model in two online experiments with 312 adults [3]. Hierarchical priors show learner-to-learner variation without overfit on sparse data. Their model selects the teacher's next example and leaves the learner's tested skill unselected. It supplies prior structure. BKT supplies the updater.

Zanellati et al. review hybrid tracers that combine Bayesian posteriors with machine learning [4]. Their taxonomy finds hybrids give better results than pure BKT and pure deep learning on correctness prediction. The surveyed hybrids predict answers on large logs. They do not guide what the learner sees next.

On the separate parameter-selection bake-off, bkt-v1 reads 0.888 against 0.889 for the running-proportion baseline. That cut differs from the shipped-params synthetic row (0.785) and the live harness (0.7397 against 0.7368) in Table 5. This project adds the control module on top.

Mai et al. combine deep temporal models with a probabilistic layer for calibrated, readable estimates [5]. The limit here is cold start. One self-directed learner arrives with almost no graded history. We keep the Bayesian updater with an informed setting from onboarding data, and mastery then drives difficulty.

Deep tracing gives less accuracy for less clarity and needs much data [6]. We reject it here. The loop needs few-shot updates a learner can read, and bkt-v1 already matches the baseline.

Taken together, these works split the tracing problem by layer. Chen et al. supply prior structure over learners. Zanellati et al. supply a taxonomy of updaters. Mai et al. supply a temporal encoder with a probabilistic head. None joins the posterior to the next task the learner sees. The planner needs that join, because a mastery estimate that never changes the plan cannot steer it. Cold start also binds the choice. A self-directed learner arrives with a few graded answers. A four-parameter Bayesian updater with priors informed at onboarding is the only arm that can estimate from the first session. We keep BKT as the updater. We read the hybrid and deep arms as evidence about accuracy, not as candidates for the shipped loop.

2.2 Generation and ranking

The STEM prompting study generates items across subjects and checks them per subject, so coverage is its claim [7]. No record ties a question to its chunk, so you cannot trace a wrong item to its source. Our harness shows groundedness 0.867 with near-duplicate rate 0.004 on cited chunks.

The lecture-to-quiz study generates local multiple-choice items with deterministic shape checks [8]. Shape checks find malformed items, but miss well-formed questions on wrong passages. Each call here makes one single-family question and records its chunk citations.

Tagging work generates knowledge-component tags from items [9]. Tags let us keep per-skill histories. The study ends with the tag and never guides practice. Each problem here has its family label from generation into the run.

Generation uses retrieved passages as its source [10]. The dense arm encodes 754 chunks at 768 dimensions [11]. The lexical arm scores the terms [12]. Reciprocal fusion combines the arms with pool 25 [13].

The reranker reorders the top 50 and increases r@1 by 7 and r@3 by 10 [14].

These works expose a common split between item quality and item provenance. The STEM study measures coverage across subjects, and the lecture-to-quiz study measures shape validity, so both check the item in isolation. The tagging study attaches a knowledge component after the item exists, so the tag inherits whatever provenance

the item had. None records which retrieved chunk produced which sentence of the stem or the key. Provenance is what lets a learner contest a bad question and lets us audit groundedness at all. One family per call makes that audit tractable. The request names the family, the response cites its chunks, and the stored citation list is the evidence. The contract therefore admits one family and one question per call. It spends calls instead of mixing formats.

2.3 Grading authority

The EDUCON study grades short answers with a rubric-driven model [15]. Rubrics make judgments repeatable. Our live matrix grades written runs 4 of 4 with real rubric grades. A rubric cannot run code, so code answers execute in a hidden-test sandbox [16].

Unlike the EDUCON rubric study, which grades after the answer, the grade-prediction study forecasts about 0.73 before scheduling [17]. We use prediction for planning only and keep grading separate, where objective runs succeed in 2 of 2 cases with exact match on authored answers. The server grade stays approved while client checks stay advisory.

The two grading studies also differ in where authority sits. A rubric model produces a judgment that a learner cannot recompute, so the rubric itself has to be the contract. A code sandbox produces a pass or fail that is reproducible from the hidden tests, so the tests have to be the contract. Neither result transfers authority to the client. We keep that split: the server owns the grade for both families, and the browser renders an advisory check that never leaves the session. The split also explains the refusal path. A coding item with no derivable input-output contract cannot be graded in the sandbox. The arm refuses that material instead of grading prose in its place.

2.4 Guidance without answers

Alhazbi et al. review 25 studies and find most valid trace indicators cover time management [18]. That establishes adherence as a metric. It does not guide a learner in a hard problem. Our ladder guides learners from nudge to hint to targeted help to gated reveal.

Uzun et al. connect feedback engagement to self-regulation and course results [19]. Readable feedback helps learning. Each ungated hint tends to give answers, so the system locks the reveal until the learner confirms.

Think-aloud work separates genuine attempts from answer-extracting moves [20]. Our replay converts 4 of 4 verdict-triggered replans after the fix, against 4 of 4 holds before it. Feedback copy stays advisory for the same reason and never changes a grade.

Adherence and engagement are necessary signals, but they measure whether the learner stayed, not whether the learner struggled productively. The think-aloud work shows that some stays are answer extraction, so a help feature can raise engagement while lowering learning. A gated ladder addresses that failure directly. The learner must confirm an attempt before a reveal, so the reveal cannot substitute for the attempt. The same logic keeps feedback copy advisory. A hint that changed a grade would make the grading arm a second authority. The audit trail would then not say

which arm decided. We prefer less help that stays auditable over more help that confounds the evidence.

2.5 Sourcing note

We built this survey through advisor-guided curation and citation chaining, not a fresh database search. We started with the tracing trio and the loop anchors: single-family generation, per-family arms, the gated ladder, and advisory feedback. One representative work covers each sub-theme. We ran no hundred-scale screening, and we claim none.

Table 1 Phase 2 literature map with selected and rejected options

Stream	Selected	Rejected	Gap it closes
Tracing	BKT bkt-v1	DKT hybrid	Mastery-blind plan
Generation	Single family grounded	Multi-family ungrounded	Untraceable gen
Ranking	Hybrid plus rerank	Dense only	Ungrounded rank
Grading	Server arm per family	Client-side grade	Single signal
Guidance	Gated ladder	Answer-giving tutor	Reveal control
Loop	Jev gate fail-closed	Raw LLM choice	Ungated replan

Generation is fluent but untraceable, and tags guide nothing, so questions cannot support per-skill practice. Grades use one signal or the client, so they cannot support plan decisions. Guidance measures adherence or reveals answers, so attempts stop being evidence.

This loop addresses all three. The system cites sources for generation, and each family has its own approved grade. Difficulty follows mastery, the ladder stays gated, feedback stays advisory, and the verdict triggers regeneration.

3 Materials and Methods

3.1 Closed loop

The loop runs five stages: generation, grading, tracing, guidance, and the verdict gate. Generation makes practice from learner materials. Grading scores each answer with the arm that handles the family. Tracing updates mastery, and guidance uses a ladder and a gated reveal.

The verdict gate decides between 'hold' and 'suggest replan'. A hold is the UI's Keep action; it writes no event. 'Suggest replan' copies an exact verdict pair (the verdict id and decision timestamp) onto the new roadmap, so the decision can be replayed.

3.2 Server gate and grading arms

One call has one family and one question, so a run expands to N calls with bounded concurrency. Objective grading uses exact rules, written grading uses a rubric, and coding grades in a sandbox.

Client checks stay advisory and never change the grade, while the coding arm refuses material it cannot ground and marks the attempt honestly.

3.3 Tracing and guidance

BKT bkt-v1 tracks mastery with released parameters. The Adaptive module uses the BKT band and stays advisory. We leave the module uncalibrated by design in this draft. The guide ladder moves learners from nudge to hint to targeted help to gated reveal. Feedback copy uses updated content only with a static fallback. Advisory text never changes a grade.

3.4 Verdict gate

Jev (jev-1.13), a typed System One decision model reached through OpenRouter, scores each decision. Sufficiency is Jev's evidence-sufficiency score in [0,1]. The router applies the 0.7 rule. The loop changes Jev failure to 'hold' through a fail-closed path. The gate approves a replan at 0.7.

The sweep across 0.5 to 0.7 agreed with the human label on 18 of 21 states. The 0.8 candidate held a live 0.79 case at the gate when we expected a replan, so we reject it. Each decision records an exact verdict pair for audit.

3.5 Comparison design

The design uses two parts. The offline part uses frozen golds with fold hashes and a fail-closed gate. The longitudinal part runs four synthetic archetypes with held-out separation and scores conversion from verdict to replan and rebound in adherence. The live browser test set stays a deployment check only.

3.6 Datasets

The retrieval corpus is the ACCA APM study text (572 pages, 754 durable chunks) with 768-d vectors and zero nulls. OULAD (the Open University Learning Analytics Dataset) was the Phase-1 calibration source with real-learner logs; those logs stay reserved and no OULAD result enters this paper [21]. The 788-chunk split remains a frozen precedent only. Generation golds contain 30 objective plus 6 written plus 4 coding items with an attempt gate of 5. The longitudinal set contains four archetypes named steady, crammer, struggling, and returning. This draft uses no new live-user dataset.

3.7 Tool stack and fixed budgets

Table 2 shows the pinned stack. We state the fixed budgets here. Generation and judging use one provider service [22]. A transformer encoder gives dense vectors [23]. The Qwen3 pair supplies embeddings and rerank from one container [14, 24].

We run reranked-754 on 30 questions times 3 arms. We run the endpoint on 30 questions times 2 flags. The second corpus contains 1094 chunks with 30 verified golds.

The probe uses 395 calls at 0.099 dollars. Jev uses at most 15 calls for the dip replay. Generation uses reasoning off with temperature 0.3 and a 30 s timeout.

Every run in this paper has a fixed budget, and the budget is part of the claim. Reranked-754 spends 30 questions times 3 arms. The endpoint check spends the same 30 questions times 2 flags. The DDIA corpus (1094 chunks from *Designing Data-Intensive Applications*) spends 30 verified golds over 1094 chunks. The generation probe spends 395 calls at 0.099 dollars across 13 arms. The structured-output probe (six reasoning tiers, 30 items each, 180 items total) runs at p50 3973 ms and p95 10547 ms, and no call exceeds the 30 s timeout. The dip replay spends at most 15 Jev calls under a stop rule and used 7. A repetition beyond these budgets is a new run and not a rerun, and we label it as such wherever a number appears.

Table 2 Pinned tool stack (v0 with versions from live config)

Part	Pin
Generation	OpenRouter deepseek/deepseek-v4-flash-0731, reasoning off, T 0.3, 30 s
Feedback	meta/muse-spark-1.3-contributor low via OpenRouter
Judge	Jev (jev-1.13) hosted API, separate key
Retrieval	Qwen3-Embedding-0.6B plus Qwen3-Reranker-0.6B on demand at :8200
Grading	Piston sandbox (ghcr.io/engineer-man/piston) at :2000, Python 3.12.0
Client	CodeMirror 6 plus Pyodide 0.27.8 advisory

3.8 Reproducibility

Three facts allow reruns. Gates fail closed by default. Each logical call has one request id from frontend to backend logs. We version evidence beside the draft with golds, harness scripts, seeds, and replay outputs.

The triple is checkable without our infrastructure. A fail-closed gate blocks a generation or a replan when its threshold is unset. A misconfigured run then cannot fall through to a plausible default and enter the results. The request id joins the frontend call, the backend handler, and the worker line. We used that join to check the live matrix's zero-error claims instead of assuming them. Versioned evidence sits beside the draft as golds, harness scripts, seeds, and replay outputs. A reader can therefore rebuild a table from stored inputs rather than trust a transcript. A mismatch between paper and code is therefore a defect we can locate, not a dispute about which build produced a number.

The scope limit belongs to the method. One learner drives the longitudinal replay, and four synthetic archetypes stand in for a classroom cohort. The retrieval and generation corpora are real documents, and the live matrix exercises the deployed stack on real requests. No live-user dataset enters the results. This design supports claims about the loop's mechanics and its measured behavior on frozen evidence. It does not support claims about population effects, long-term retention, or classroom deployment. No result here is read past the design that produced it.

4 Results and Discussion

4.1 Retrieval and ranking

Table 3 and Figure 1 show the coherent 754 set. Recall at k (r@k) is the share of questions whose answer chunk ranks in the top k; mean reciprocal rank (MRR) averages the reciprocal of the answer rank. Hybrid plus rerank increases r@1 from 0.60 to 0.67 and r@3 from 0.80 to 0.90. MRR increases from 0.720 to 0.797. Dense and hybrid reproduce the recorded in-process baselines, which validates the harness. The rerank arm is then compared against that validated harness.

Table 3 Retrieval on the frozen 754-chunk corpus (30 verified questions, coherent headline)

Arm	r@1	r@3	r@5	MRR
Dense	0.60	0.77	0.87	0.706
Hybrid	0.60	0.80	0.87	0.720
Hybrid plus rerank	0.67	0.90	0.93	0.797

Note: frozen 788-split precedent is 0.70/0.83/0.90/0.788 dense, 0.73/0.90/0.93/0.816 hybrid, and 0.77/0.97/0.97/0.858 reranked. It is superseded for the headline.

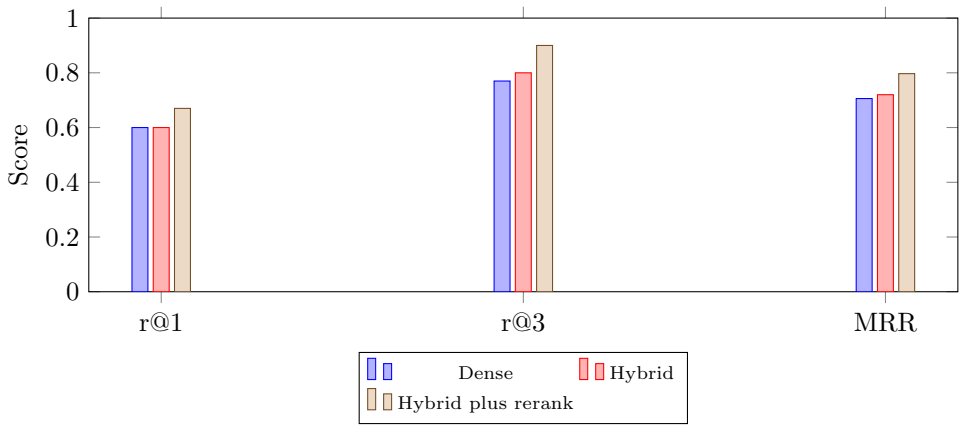


Fig. 1 Retrieval on the frozen 754-chunk corpus (30 verified questions): hybrid plus rerank lifts r@1 from 0.60 to 0.67 and r@3 from 0.80 to 0.90. DDIA second corpus (1094 chunks) confirms the direction with gains of +16 at r@1, +17 at r@3, and +0.139 MRR.

The plain endpoint reproduces the live RPC probe (0.60/0.73/0.87/0.703), not the in-process harness. The reranked endpoint reads 0.70/0.90/0.93/0.813, slightly above the harness arm (0.67/0.90/0.93/0.797) because the RPC candidate set differs; we report the endpoint row as the deployed number. The second DDIA corpus shows the same trend with gains of +16 at r@1, +17 at r@3, and +0.139 MRR.

Title-prefix gives no improvement. We reject INT8 collapse to 0.10 as an arm.

The two corpora agree on the shape of the result. Fusion alone does not move r@1 on the 754 set, where dense and hybrid both sit at 0.60. The lexical arm adds recall depth without changing the first hit. Rerank moves both the first hit and the depth. A cross-encoder trained on query-passage pairs should do that when the candidate pool already contains the gold. The r@3 gain is larger than the r@1 gain in both corpora. The reranker therefore recovers near misses more often than it promotes a single buried gold. The live route preserves the ordering between plain and reranked search on every reported metric, so the HTTP boundary does not add a retrieval defect. We report the 754 table as the headline and keep the 788 split as a precedent. The 788 numbers came from a retired chunker and are not comparable to the deployed corpus.

4.2 Generation quality

Table 4 shows the harness. The structured-output probe over 180 items maintains schema validity at 0.9611 and citation validity at 0.9133. Gold support is 0.7803 and measures a different fact from groundedness.

Near-duplicate rate is 0.0044. The fresh live judge shows groundedness at 0.8667 (26 of 30). Coding derivability is 0.5349 and passes the 0.5 gate. Gold agreement is 3 of 3 coding golds with an authored accepted value (4 coding golds total).

Table 4 Generation quality (structured-output probe n=180 with judge n=30 fresh live; coding gate 0.5)

Metric	Value
Schema valid	0.9611
Citation valid	0.9133
Gold support	0.7803
Near duplicate	0.0044
Cost	$0.061289
Groundedness judge	0.8667 (26/30)
Coding derivable	0.5349

Schema validity is 0.9611 and citation validity is 0.9133. The contract checks pass on most items, and citation validity is the weaker of the two rates. Gold support at 0.7803 is lower than the judge's groundedness at 0.8667 (26 of 30 fresh items) because the two ask different questions. Support tests whether a cited chunk contains the authored answer. Groundedness asks whether the item is supported by any cited chunk. We report both and never average them. Coding derivability at 0.5349 is the weakest headline number. It passes the 0.5 gate by a narrow margin, so the coding arm is the first place a fresh corpus would likely fail. Cost at $0.061289 for the structured-output run keeps the harness cheap enough to repeat on a new corpus. That matters more for a single-learner system than a tighter point estimate on one corpus.

4.3 Knowledge tracing

Table 5 and Figure 2 show three populations with their scope. The synthetic bake-off establishes the prior, the live harness measures real attempts, and the external

anchor sets the 0.6 baseline. The result supports parity. We leave difficulty calibration unmeasured, so the module remains advisory.

Table 5 BKT tracing by population (cite scope with each use)

Population	AUC	ECE
Synthetic bake-off (shipped params)	0.785	0.057
Live harness (282 obs, 139 seqs)	0.7397	0.1401
Running proportion live	0.7368	0.2679
External anchor	0.60-0.61	–

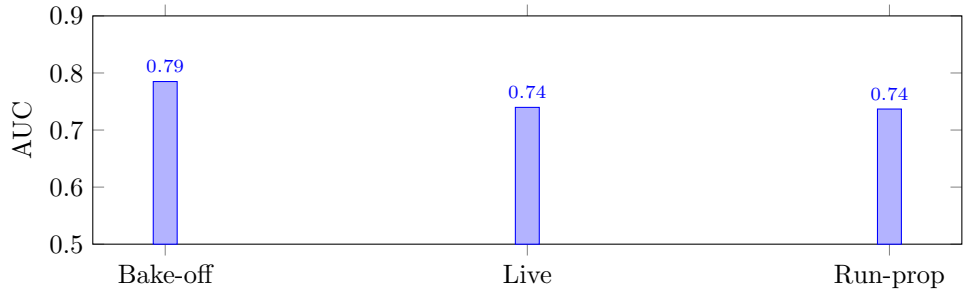


Fig. 2 BKT AUC by population with the external 0.60-0.61 anchor as the parity baseline. The y-axis starts at 0.5. Live ECE is 0.1401 against 0.057 bake-off and 0.2679 running-proportion. Difficulty calibration stays unmeasured, so the Adaptive module remains advisory.

The three populations answer three different questions. The bake-off says the updater recovers a known synthetic mastery process, at AUC 0.785 and ECE 0.057. The live harness says the same updater ranks real attempts about as well as a running proportion, at 0.7397 against 0.7368. It also cuts calibration error roughly in half, at 0.1401 against 0.2679. The external anchor sets the 0.602-0.611 floor; both arms clear it. The anchor's ECE is not reportable, so this is a floor check, not a like-for-like calibration comparison. None of these rows measures whether the difficulty band is correct, because the eligible pool held one question. The Adaptive module therefore stays advisory in this draft, and the paper claims no difficulty effect.

4.4 Closed loop verdicts

Table 6 and Figure 3 show the verdict table. The post-fix replay changes 4 of 4 'suggest replan' verdicts to roadmaps. All four use extend-deadline with an exact verdict copy. Pre-fix showed 4 of 4 at low confidence on span-zero inputs. The dip replay holds 4 of 4 at the September setting and blocks the struggling case at the September-12 setting.

Post-fix, every archetype reaches confidence 1.0 and converts. Every conversion carries the exact verdict pair, so an auditor can replay the decision from the stored payload without reading a transcript. The dip replay shows the gate working at the

Table 6 Verdict-gated replans across four archetypes (all extend-deadline; rebound sufficiency is a cross-user range)

Archetype	Window	Verdict	Conf	Sufficiency	Converted
Steady	Rebound (Oct-14)	suggest replan	1.00	0.66-0.77	Yes
Crammer	Rebound (Oct-14)	suggest replan	1.00	0.66-0.77	Yes
Struggling	Rebound (Oct-14)	suggest replan	1.00	0.66-0.77	Yes
Returning	Rebound (Oct-14)	suggest replan	1.00	0.66-0.77	Yes
Steady	Trough (Sep-5)	hold	0.33	0.76	No
Crammer	Trough (Sep-5)	hold	0.30	0.75	No
Struggling	Trough (Sep-5)	hold	0.28	0.77	No
Returning	Trough (Sep-5)	hold	0.35	0.66	No
Struggling	Week-4 (Sep-12)	hold (gate)	0.61-0.69	0.77	No

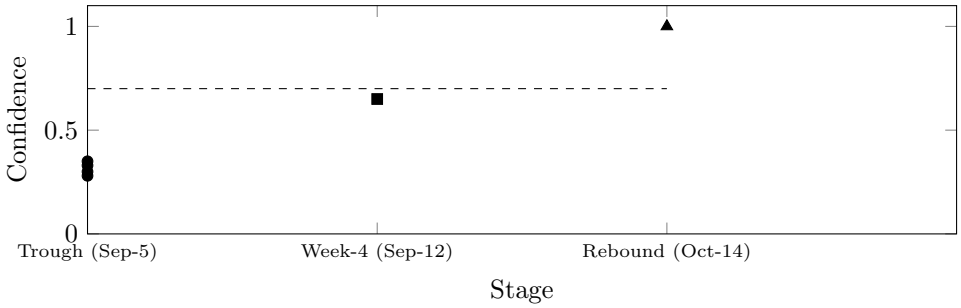


Fig. 3 Verdict confidence across the dip window with the 0.7 approve gate (dashed). Trough holds 4 of 4 at 0.28-0.35; week-4 struggling reaches 0.61-0.69 but stays gate-held; rebound converts 4 of 4 at 1.0. All conversions use extend-deadline with an exact verdict copy.

boundary. At the September trough all four cases hold at 0.28-0.35. At week four the struggling case reaches 0.61-0.69 and still stays below the 0.7 gate. At the rebound it converts at 1.0. That gradient is evidence that the gate tracks the evidence rather than the calendar. The pre-fix 4 of 4 holds at 0.18-0.34 came from span-zero inputs. We report them only as the defect the fix removed, not as a second result. The adherence arm produced 24 rows with next-week adherence at 0.000. That capture lands in the rebound era and not at a week-three dip. We therefore keep it in the appendix and claim no adherence effect.

We select the verdict-gated loop as the reported system. It connects grounded generation, server grading, BKT tracing, and reranked retrieval to a measured replan decision. The live test set allows deployment only. It shows zero console errors across objectives, written rubrics, coding grades, guides, practice runs, feedback copy, mastery reads, and ingestion.

4.5 Architecture figures

Figure 4 shows the deployment with its sidecars. Figure 5 shows the five-stage loop with its gate. Figure 6 shows the two-part comparison, which keeps the deployment check separate.

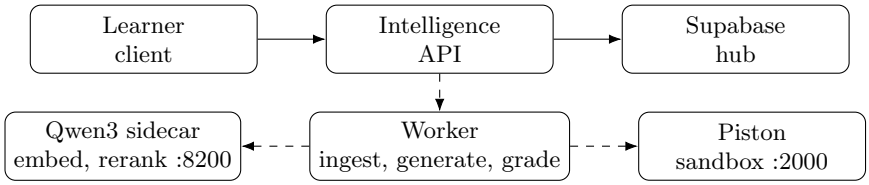


Fig. 4 Deployment map: solid edges are requests, dashed edges are worker calls

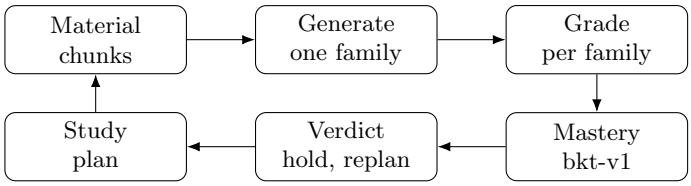


Fig. 5 Closed loop with verdict gate

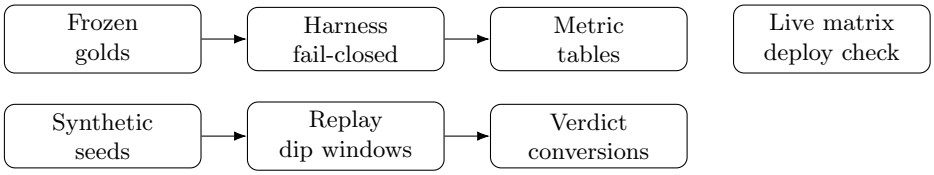


Fig. 6 Two-leg comparison pipeline with the deployment check kept separate

4.6 Rejected arms and thresholds

Title-prefix gives no improvement on the frozen split, and INT8 quantization reduces retrieval to 0.10 for a 2.45x speedup, so we reject both. The 0.8 candidate held a live 0.79 case at the gate when a replan was expected, so we keep 0.7. We reject deep tracing, multi-family calls, client-side grades, and answer-giving tutors (see Table 1).

4.7 Parity and nulls

BKT matches the baseline on rank order and halves its calibration error. That is an equal result with better calibration, not an improvement. Phase I measured change detection as a robust null; Phase II ships no detector. We leave split-conformal intervals unshipped. We leave difficulty calibration unmeasured, so the Adaptive module remains advisory.

4.8 Deployment check

The live test set is a deployment check that never adds a number to results, where objective runs succeed in 2 of 2 cases and written rubrics succeed in 4 of 4 cases. Coding grades succeed in 3 of 3 cases and guides succeed in 2 of 2 cases. Practice runs succeed in 2 of 2 cases, feedback checks succeed in 3 of 3 cases, and mastery checks succeed in 2 of 2 cases.

Ingestion completes and the system logs skips. Zero console errors appear across viewports, and the full-book run becomes ready in about 88 seconds and we verify cleanup. The loop uses a managed stack with the sidecar on demand.

5 Conclusion

The verdict-gated loop joins what prior planners keep apart: server grading per answer family, BKT tracing, reranked retrieval, and a gate that refuses what it cannot ground. The evidence supports parity on ranking with better calibration and 4 of 4 measured replans on synthetic histories; it does not support population effects or long-term retention. The test that would change the claim is a real-learner study with held-out validation. Code is available at https://github.com/rings0fsaturn/study-planner.

Supplementary information. Supplementary files contain the eight versioned evidence records behind Appendix A: reranked-754, endpoint retrieval, the second DDIA corpus with questions, synthetic data generation emission, the fresh generation harness (summary, report, live), the difficulty baseline, dip replay, and full-book timing.

Acknowledgments. Prof. Ramesh Prakash Guledgudd guides this work.

Declarations

- Funding: Not applicable.
- Conflict of interest/Competing interests: The authors declare no competing interests.
- Ethics approval: Not applicable.
- Consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Availability of data and materials: The frozen 754-chunk corpus with golds plus four synthetic seeds allows reruns.
- Code availability: Code is available at https://github.com/rings0fsaturn/study-planner.
- Authors' contributions: R.S. built the loop and collected the evidence. R.P.G. guided the work.

Appendix A Evidence index

Table 3 and Figure 1 map to `research/doc/2026-10-22-reranked-754.md` plus its JSON (30 verified questions, 0 skipped, dense and hybrid arms reproduce baselines exactly). The endpoint values map to `research/doc/2026-10-22-endpoint-retrieval.md` plus its JSON (plain 0.60/0.73/0.87/0.703, reranked 0.70/0.90/0.93/0.813). The DDIA confirmation maps to `research/doc/2026-10-22-second-corpus-ddia.md` plus questions and JSON (1094 chunks, 0 NULL, 30 verified golds). Table 4 maps to `research/doc/generation-quality-harness/summary.json` (structured-output probe n=180) plus `live.json` (fresh 30-item judge,

`liveSource: live`). Table 5 and Figure 2 map to the harness live output (282 obs over 139 skill sequences) with shipped parameters in `packages/py-progress/src/py_progress/mastery.py`. Table 6 and Figure 3 map to `research/doc/2026-10-22-synth-extract-emission.json` (24 adherence rows, 4 of 4 converted). They also map to `research/doc/2026-10-22-dip-window-replay.md` (Jev 7 of 15 approved calls used, cloud untouched). Full-book timing maps to `research/doc/2026-10-22-fullbook-e2e-timing.md` plus its JSON (ready in about 88 s wall, material deleted, card-gone verified).

References

[1] Bloom, B.S.: The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. Educational Researcher **13**(6), 4–16 (1984)

[2] Corbett, A.T., Anderson, J.R.: Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction **4**(4), 253–278 (1994)

[3] Chen, A.M., Palacci, A., Vélez, N., Hawkins, R.D., Gershman, S.J.: A hierarchical Bayesian model of adaptive teaching. Cognitive Science **48**(7), 13477 (2024) https://doi.org/10.1111/cogs.13477

[4] Zanellati, A., Di Mitri, D., Gabbrielli, M., Levrini, O.: Hybrid models for knowledge tracing: A systematic literature review. IEEE Transactions on Learning Technologies **17**, 1021–1036 (2024) https://doi.org/10.1109/TLT.2023.3348690

[5] Mai, N.T., Cao, W., Liu, W.: Interpretable knowledge tracing via transformer–bayesian hybrid networks: Learning temporal dependencies and causal structures in educational data. Applied Sciences **15**(17), 9605 (2025) https://doi.org/10.3390/app15179605

[6] Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L.J., Sohl-Dickstein, J.: Deep knowledge tracing. In: Advances in Neural Information Processing Systems, vol. 28 (2015)

[7] Chan, K.W., Ali, F., Park, J., Sham, K.S.B., Tan, E.Y.T., Chong, F.W.C., Qian, K., Sze, G.K.: Automatic item generation in various STEM subjects using large language model prompting. Computers and Education: Artificial Intelligence **8**, 100344 (2025) https://doi.org/10.1016/j.caeai.2024.100344

[8] Shintani, S.A.: Self-hosted Lecture-to-Quiz: Local LLM MCQ Generation with Deterministic Quality Control. arXiv:2603.08729 (2026)

[9] Moore, S., Schmucker, R., Mitchell, T., Stamper, J.: Automated generation and tagging of knowledge components from multiple-choice questions. In: Proc. ACM Learning @ Scale (L@S), pp. 122–133 (2024). https://doi.org/10.1145/3657604.3662030

[10] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, vol. 33 (2020)

[11] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.: Dense passage retrieval for open-domain question answering. In: Proc. EMNLP, pp. 6769–6781 (2020). https://doi.org/10.18653/v1/2020.emnlp-main.550

[12] Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval **3**(4), 333–389 (2009)

[13] Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval, pp. 75–82 (2009). https://doi.org/10.1145/1571941.1571951

[14] Qwen Team: Qwen3-Reranker-0.6B. Model card, Hugging Face, https://huggingface.co/Qwen/Qwen3-Reranker-0.6B; accessed 2026-10-23. Cross-encoder reranker in this work; 0.6B parameters, 32k context (2025)

[15] Duong, T.N.B., Meng, C.: Automatic grading of short answers using large language models in software engineering courses. In: Proc. IEEE Global Engineering Education Conference (EDUCON), pp. 1–10 (2024). https://doi.org/10.1109/EDUCON60312.2024.10578839

[16] EngineerMan: Piston: A high performance general purpose code execution engine. GitHub repository, https://github.com/engineer-man/piston. Sandbox used for the coding grading arm; image `ghcr.io/engineer-man/piston` with Python 3.12.0 runtime; accessed 2026-10-23 (2018)

[17] Islam, M.S., Mukaddes, A.M.M., Uddin, M.: Predict students' performance and optimize preparatory leave: An integrated deep learning and dynamic programming approach. In: Proc. International Conference on Computer and Information Technology (ICCIT) (2024). https://doi.org/10.1109/ICCIT64611.2024.11021881

[18] Alhazbi, S., Al-Ali, A., Tabassum, A., Al-Ali, A.K., Al-Emadi, A., Khattab, T.M., Hasan, M.A.: Using learning analytics to measure self-regulated learning: A systematic review of empirical studies in higher education. Journal of Computer Assisted Learning **40**(4), 1658–1674 (2024) https://doi.org/10.1111/jcal.12982

[19] Uzun, Y., Suraworachet, W., Zhou, Q., Gauthier, A., Cukurova, M.: Engagement with analytics feedback and its relationship to self-regulated learning competence and course performance. International Journal of Educational Technology in Higher Education **22**(1), 17 (2025) https://doi.org/10.1186/s41239-025-00515-3

[20] Zhang, J., Borchers, C., Wang, C., Kumar, V., Teffera, L., McLaren, B.M., Baker, R.S.: Understanding gaming the system by analyzing self-regulated learning in think-aloud protocols. In: Proc. International Learning Analytics and Knowledge Conference (LAK) (2026). https://doi.org/10.1145/3785022.3785025

[21] Kuzilek, M., Hlosta, M., Zdrahal, Z.: Open University Learning Analytics dataset. Scientific Data **4**, 170171 (2017)

[22] OpenRouter: OpenRouter: Unified API for language models. Provider documentation, https://openrouter.ai/docs; accessed 2026-10-23. Generation and judging run through this provider (2025)

[23] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)

[24] Qwen Team: Qwen3-Embedding-0.6B. Model card, Hugging Face, https://huggingface.co/Qwen/Qwen3-Embedding-0.6B; accessed 2026-10-23. Dense encoder in this work; 0.6B parameters, 32k context, up to 1024 dimensions with MRL (2025)